1 Repository Client

You use the MPDV Repository Client MRC to display and edit repository data. It provides a user-friendly access.

1.1 Quick start

This section provides a quick overview of how to work with the Repository Client. The individual steps are only briefly described. For further information on the individual steps, refer to the sections in the following, if required.

Installation

Requirements

To use the Repository Client, you must have installed the Microsoft DotNet framework (at least version 4.5.2).

Program installation

To install the program, just copy the folder including the binary files into your system. An installation program is not required.

Installation of developer license

If the developer license is not available, you can only read the data. You cannot save or export the data.

The developer license is handed out once you have attended a respective Customizing Training. The developer license is provided as *.lic file. This file is included in the data medium that you have received during the training: Folder "Repository Client", subfolder "`tools/licence`", e.g. `x:\CUT-MOC_81_files\Tools\MPDVRepositoryClient\tools\licence\mpdvWrite.lic`.

Copy the folder "`licence`" with its content into the folder of the Repository Client in the roaming directory of the Windows user, e.g.:
`C:\Users\%User%\AppData\Roaming\MPDV\RepositoryClient\licence\mpdvWrite.lic`
This folder is automatically created on the first start of the Repository Client.

Before you start

Before you start working with the Repository Client, you must make sure that the Repository Client has been installed according to the installation instructions and that the required license files are stored as described there.

In general, the repository is empty when you start work. However, if you do not want to start with an empty repository, it is recommended to make sure that the data you want to work with is available.

First steps

Start the Repository Client.

➔ Start .\bin\mrc.exe

Now load or create a Workset.

A workset specifies the sources of the repository that you want to edit.

➔ Click the button *Load work set* in the file-based repository
-> select workingset.work

Click the button *Repository* → *Load Repository* to load the data from the sources specified in the workset.

How to proceed further depends on what you want to do via the Repository Client.

Tip: Working with perspectives

The Repository Client provides the possibility to use different perspectives for different tasks. Do not hesitate to close views you do not need or drag them to another open view in order to tab them and hence provide for more space and clarity. You can also use more than one view of the same type. Simply adapt your perspective to the requirements of your tasks.

Example:
If you create a service and you would like to check with an existing service how to populate the fields, you can simply open another service view. Thus, you do not need to destroy your current view and re-orient yourself later.

Default perspectives

The installation of the Repository Client provides default perspectives:

default

This is a good perspective to start with. It provides a combined view for server and client-related contents.

Select a domain on the top left. Via the included relations, the top right area shows the services, servicesGUI and properties of this domain. The bottom right area shows the ServiceParameters of the service selected above, the ServiceParameterGui of the ServiceGui selected above and the ControlDataSources, ReferenceData and Authorizations of the selected domain.

default client

Similar to the "default" perspective but limited to the contents concerning client development.

default server

Similar to the "default" perspective but limited to the contents concerning server development.

DB schema

Shows documentation of the database structure.

Validation

Use this perspective to validate contents.

Proceed as follows:

- Open perspective and load data from workset.
- Perform validation: tab *Repository* --> button *Validate*.
- A CSV file listing the identified irregularities is generated in the sub-directory "validation_logs" of the Repository Client's installation directory. The application that is linked to this file type in the operating system opens the CSV file.
- In the views for Domains, Properties, ServiceParameters, etc. the Repository Client only shows the entries with detected irregularities.

You should analyze and, if necessary, correct the detected irregularities. Not every irregularity leads to an error.

1.2 Start and exit Repository Client

Start the Repository Client via the Windows start menu, a link on the desktop or the command line. As soon as all required components have been loaded, the application window is shown.

You can start and run the Repository Client multiple times in parallel on a PC. You can access different repository data in each of the started instances of the Repository Client. For example, you can view several versions of the repository at the same time.

You can also start the client by opening one of the workset files. On start of the client, the system attempts to load the contents of the workset defined in this file. This option is available after the first start of the Repository Client.

Command line parameters

You may transfer parameters to the Repository Client upon the start. The following parameters are supported:

`-perspective/-p <perspectivename>` Use this parameter to start the Repository Client with a specific perspective. If you do not use this parameter, the last active perspective is started by default.

`-workset/-w <worksetfile>` Use this parameter to specify the workset to be loaded. If you do not specify the workset on start of the application, the last loaded workset is loaded.

`--autoload/--a` If you use this parameter, the Repository Client loads the repository defined in the last active workset or in the workset transferred via parameters. This repository is loaded directly on start of the application.

`--trim/--t` If you use this parameter, the Repository Client removes so-called leading and trailing "whitespaces" when loading the repository. Only select this option if needed, because the load time increases extremely.

Exit

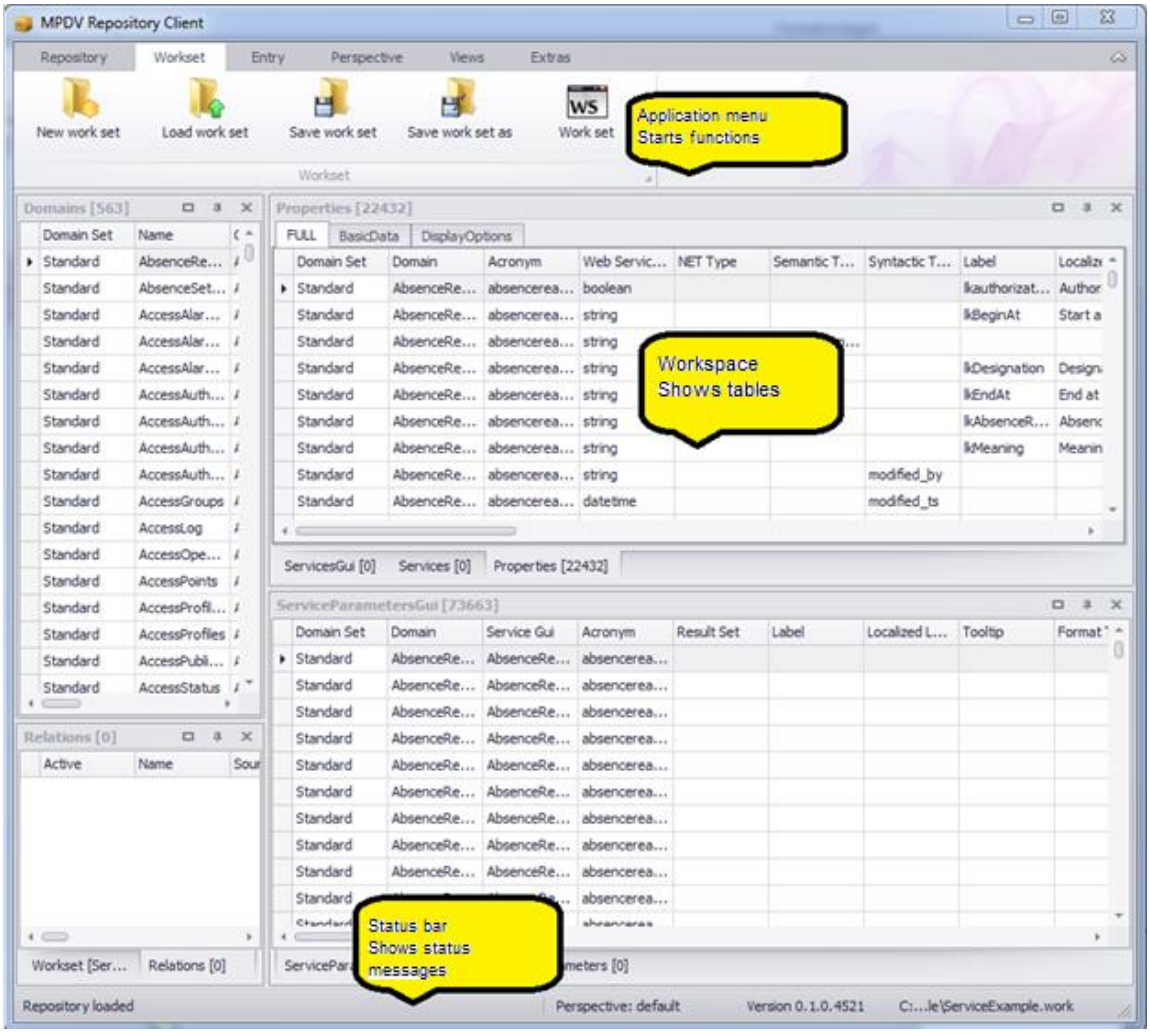
To exit the application, click  in the title bar.

If the loaded repository includes active changes when you exit the application, a respective message is issued asking you to save the changes. If you do not want to save the changes, you can discard the changes or stop exiting the application.

Note: Changes to the workset and perspective are discarded when you exit the application, if you have not saved the changes.

1.3 The Application Window

The application window forms a framework for the display of different tables. It includes the application menu with control elements to call and control different functionalities. The menu is on top of the window. A status bar is at the bottom of the window. The status bar shows progress and event messages.



You can individually dock the grids/table views. To do so, click the title bar of a table view/grid and drag it out of the docking position. For orientation purposes, the system shows the docking positions where you can drop the table view. You can also drop a table view without docking it.

1.4 Grids/table views

Grids/table views are components to present data records in a table. You can change the tables in the Repository Client according to your requirements. For each grid/table view, the functions described below are available.

The settings, that you make in a table, are saved with the perspective. To undo changes, you can switch to the standard perspective (in the application menu: *Perspective* → *Change perspective*).

Sort table data

Click the table header to sort table data in descending order. If you click the table header once more, data is sorted in ascending order. The selected sorting option is shown.

You can sort data by several columns: Press the *Shift* key of your keyboard after sorting the first column. Then click the other column headings by which you want to sort.

You can also use the context menu of the table header to sort data.

Group data in the table

You can group table data if the *group by* area is shown. If the *group by* area is not shown, you can show the area via the context menu of the table header (*Show/Hide group by box*). To group by a column, click the column header and drag it to the grouping pane. Multiple grouping is also supported.

Optimum column width (best fit)

Select the option *"Best fit"* in the context menu of the table header to adjust the column width of the selected column to the optimum width. In this case, ”optimum” means that the column is as wide as the largest entry in the selected column.

Optimum column width (all columns) / Best fit (all columns)

Click this function to adjust all columns to the optimum width.

Change column width

You can also change the column width using the mouse, i.e. move the space between two cells to the left or right.

Show and/or hide columns and entire categories

Use the context menu function *Select columns* to show and/or hide individual columns and entire categories. For this purpose, select the function in the context menu and then drag the required columns and/or categories from the table to the pool or from the pool to the table.

Change the sequence of columns and categories

Also use the mouse to change the display order of columns and categories. To do so, drag the column or category you want to move and drop it at the required location. The system will indicate the location to which the column and/or category will be allocated when you release the left mouse button.

Freezing columns to prevent horizontal scrolling

You can freeze columns at the left and right-hand side to keep the columns in view while scrolling. These column settings are included in the perspective and can be saved with the perspective. Right-click the column header and press one of the below-mentioned shortcuts to freeze columns:

- CTRL + right click: freeze at the left-hand side.
- ALT + right click: freeze at the right-hand side.
- SHIFT + right click: Unfreeze.

Filter table data

Click the filter icon in the column where you intend to set the filter and select the required filter option from the list; i.e. you select one of the values available in the table or compose a combination of values in the *user-defined* filter.

You can also set several filters in different columns. The table footer indicates that the table has been filtered and also shows the filter criteria. Select the function *Edit filter* on the right of the footer to open the filter editor. Use the filter editor to create complex filter criteria across all columns. You may also open the filter editor via the context menu of the table header.

Search box

Use the context menu of the table header to access the option *Show search box.* This option provides a search box within the table. Use this box to quickly search and/or filter the requested data. Simply start typing in this box and the system will only show those rows matching the data you typed. The more characters you enter, the more you narrow down the result.

Filter row

Use the context menu of the table header to open the option *Show filter row*. This option provides an additional row shown below the table header. You can enter a search term in any column, and the system will narrow down the displayed rows appropriately. The system supports wildcards. You can also combine search terms in various columns to restrict the search result.

Edit rows

Double-click a row or press the "Enter" button to switch to the editing mode and edit the respective row. When you have finished editing and leave the row, the editing mode is terminated. Edited rows are highlighted in color.

1.5 The application menu

You can use the application ribbon menu of the Repository Client to control various functions of the tool. It includes several tabs that are described in the following.

Workset

Includes functions to administer worksets. A workset specifies the sources included in the repository that you want to edit. To display a workset, use the workset panel which consists of a grid/table view.

- **New workset**: This function creates a new workset. If a workset is loaded that has been modified, a dialog pops up asking you to save the changes.
  Click "Yes" to save the changes, "No" will discard them. In both cases, a new workset is created. Click "Cancel" to cancel the process of creating a new workset.
- **Load workset**: This functions loads a workset from an existing file. You can select the workset to be loaded via a file dialog. If the currently loaded workset has been modified, you can save the changes as described above.
- **Save workset**: This function saves the current workset in the file from which it was loaded. If the current workset is new, a file dialog pops up asking you to select the file in which you want to save the workset.
- **Save workset in**: Use this function to save the currently loaded workset in a file. You can select the files using a file dialog.
- **Workset**: Use this button to show and/or hide the grid/table view presenting the currently loaded workset. For details on the workset table, please refer to section Workset.

Repository

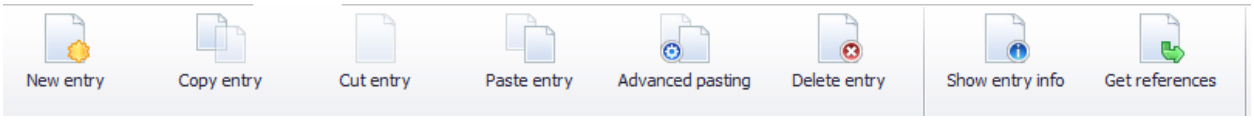
Includes functions to load and save data in the repository. In addition, you may display repository-specific data here.

- **Load repository**: Use this function to load the repository. The currently loaded workset specifies the data sources that are used to load the repository. If a repository is loaded that has been modified, a dialog pops up asking you to save the changes. Click "Yes" to save the changes, "No" will discard them. In both cases, the repository will be reloaded subsequently. Click "Cancel" to cancel the process of loading a repository.
- **Save repository:** ***only available if used in development mode**
  Use this function to save changes in the repository. If no changes have been made, an appropriate note will be displayed.
- **Export repository**: ***only available if used in development mode**
  In contrast to saving the repository, you can use this function to export parts of the repository. For details on this function, please refer to section **Error! Reference source not found.**.
- **Validate**: ***only available if used in development mode**
  You can use this function to validate your data records manually. (See section "Validation").
- **Value list**: Use this menu entry to show and/or hide the value list. The list includes permissible entries for specific fields of the repository.
- **References**: Use this entry to show and/or hide the table with repository references. For details on references, please refer to section References.

- **Changes**: ***only available if used in development mode**
  Use this button to show and/or hide the change view. This view shows the current modifications in the loaded repository.
- **Service documentation**
  Use this button to show the extended documentation of selected standard services. For further information on the service documentation, refer to section "1.9 Service documentation".

Data collection

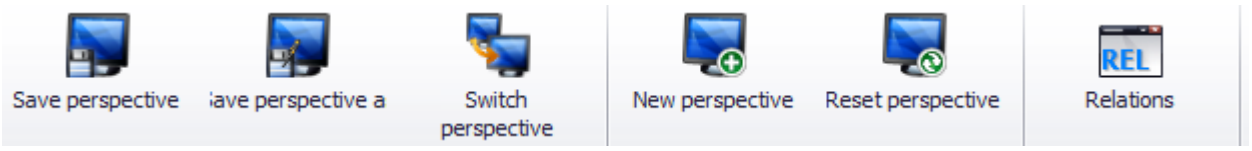
The *Entry* tab summarizes the functions that you can use to edit the loaded repository. The entries refer to the currently focused table view/grid.



- **New entry**: Use this function to create a new entry. For details on this function, please refer to *Context menu* → *New*.
- **Copy entry**: Use this function to copy selected table entries. For details on this function, please refer to *Context menu* → *Copy*.
- **Cut entry**: Use this function to cut selected table entries. For details on this function, please refer to *Context menu* → *Cut*.
- **Paste entry**: Use this function to insert (paste) entries from the cache/clipboard. For details on this function, please refer to *Context menu* → *Insert*.
- **Advanced pasting**: Use this function to edit entries in the clipboard prior to inserting them. For details on this function, please refer to *Context menu* → *Advanced pasting*.
- **Delete entry**: Use this function to delete the selected entries. For details on this function, please refer to *Context menu* → *Delete*.
- **Show entry info**: Use this function to open a dialog showing information on the currently selected entry. For details on this function, please refer to *Context menu* → *Info*.
- **Get references**: Use this function to open a new grid/table view showing the currently selected data record including referenced values. For details on this function, please refer to *Context menu* → *Get references*.

Perspective

These entries of the menu refer to the administration of perspectives. A perspective is a layout of table views/grids and includes also the associated relations between table views/grids.



- **Save perspective**: Use this function to save the currently shown arrangement of tables.
- **Save perspective as**: Use this function to save the currently shown perspective under a different name. You can enter the new name in the displayed dialog.
- **Switch perspective**: Use this function to change the perspective. A dialog with all available perspectives is shown.
- **New perspective**: Use this function to create a new perspective. Similar to the "Save perspective as" function, you can select the name of the perspective in a dialog. After entering the name, you can immediately switch to the new perspective.
- **Reset perspective**: Use this function to reset the current perspective to the status saved last.
- **Relations**: Use this menu entry to show/hide the grid/table view showing the relations between the grids/table views. For details on relations, please refer to section Relations.

Note: Changes to the perspective are discarded when you exit the application, if you have not saved the changes explicitly.

Views

Use these entries to open table views/grids. The entries will open a new grid/table view each showing the relevant data records of the repository.

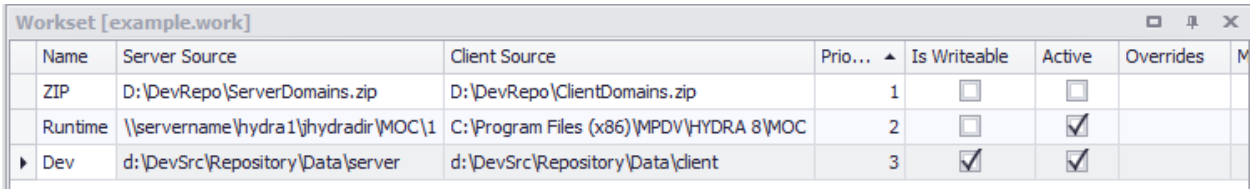
For clear identification of the data records shown in the tables, the *Parent* column is included in each of the tables. The *Parent* column includes the identifier for the father node in the repository tree. The other columns of these tables are defined by the repository documentation.

The *View* area additionally includes a group with entries for the remaining grids/table views of the application.

1.6 Workset

Worksets define a set of data sources that make up the repository that you want to edit. Use the workset management function to organize your work on different projects and create an appropriate workset for each of your projects. You can show/hide the workset table via the application menu (*Workset* → *Workset*).

Note: The workset loaded last will be loaded on start of the Repository Client.



The workset table includes the following columns:

Name

You can specify the domain set that you want to use to load the data source. The repository data include the information on the domain set that was used to load the data. You can copy data from a domain set into another domain set. Example: Copy existing applications from the domain set "Runtime" (read only) into your development directory, e.g. the domain set "Dev" (writable) where you can make changes.

Client Source

You can specify the data source that you want to use to load client data. The following options are available:

- Load data from the runtime structure of the client reference in the server
  In the server, the client configurations are stored as client reference with runtime structure. You can load the repository data from this structure.
  Example:
  HYDRA: x:\jhydradir\MaintenanceManager\rt\client\MOC
  MIP: x:\wsp_config\MaintenanceManager\rt\client\MOC
  The access is read-only.
- Load data from the runtime structure of a local MOC client
  If you enter a path to an MOC installation directory, the respective client data are directly loaded from the MOC runtime installation.
  Example:
  C:\Program Files (x86)\MPDV\HYDRA 8\MOC
  The access is read-only.
- Load data from a local directory with domain structure
  You use local directories with domain structure for the administration of your own developments. Using the Repository Client, you can read data in this directory and you can also save data into this directory.
  Example: d:\DevSrc\Repository\Data\client
- Load data from a ZIP archive
  You can enter a ZIP file (including path) that includes the data in domain structure. MPDV provides the ZIP archives as part of the trainings or on the support portal.
  The access is read-only.

Server Source

You can specify the server source that you want to use to load the server-specific data. The following options are available:

- Load data from the runtime structure in the server
  You can read the configurations for the server in a server installation. To this end, the configuration directory of the web service provider (WSP) up to the instance number is specified.
  Example:
  HYDRA: \\<servername>\<install_dir>\jhydradir\MOC\1
  MIP: \\<servername>\<install_dir>\jdir\MOC\1
  The configuration is loaded from the standard scope by default As an alternative, you can also load the configuration from the *custom* scope, if you enter the value "custom" in the field "name".
  The access is read-only.
- Load data from a local directory with domain structure
  You use local directories with domain structure for the administration of your own developments. Using the Repository Client, you can read data in this directory and you can also save data into this directory.
  Example: d:\DevSrc\Repository\Data\server
- Load data from a ZIP archive
  You can enter a ZIP file (including path) that includes the data in domain structure. MPDV provides the ZIP archives as part of the trainings or on the support portal.
  The access is read-only.

Priority

The priority of a data record specifies the loading sequence of sources that are allocated to the same data record. In the above example, data is first read from the local development directory "d:\DevSrc\Repository" and then from the runtime installations "Server" and "Client". Data records with low priority are overridden and consequently not loaded.

Is Writeable

In this column you can specify if a data source grants write access. You only require write access in workset entries where you want to make local developments.

Please note that ZIP archives do not grant write access. For this reasons, you must not enable "is Writeable" in case of a ZIP data source.

Please note that it is not supported to save data in the runtime structure (client directory or server runtime directory). You must therefore not enable "Is Writable" for data sources with runtime structure.

Overrides

You can specify in this column which domain set is overridden by the current one. This affects the resolving of references (for details please refer to section References).

An entry in the "Overrides" column does not have any effect on the loading of the data sources. See column "Priority".

Active

Use this option to enable or disable an entry.

1.7 Relations

Relations are a property of perspectives and define table filters. Tables that include relations are dynamically adapted to the selected values of another table by setting a filter. For example: Using relations, you can specify that only the service parameters of the service currently selected in another service view are displayed in a service parameter view. The *Relations* table lists the relations of the current perspective. You can call this table via the application menu (*Perspective* →*Relations*).

Relations [13]

Active	Name	Source	Target	Filter	Var0	Var1	Var2
☑	Authorizations for Domain	Domains	Authorizations	[Parent] = '$id->Authorizations'			
☑	ControlDataSources for Domain	Domains	ControlDataSources	[Parent] = '$id->ControlDataSources'			
☑	DataObject for Service	Services	Dataobjects	[DomainSet] = '$0' and [Domain] = '$1' AND [Name] = '$2'	DomainSet	Domain	Name
☑	Properties for Domain	Domains	Properties	[Parent] = '$id->Properties'			
☑	ReferenceData for Domain	Domains	ReferenceData	[Parent] = '$id->ReferenceData'			
☑	SchemaColumns for ServiceParameter	ServiceParameters	SchemaColumns	([TableName] = '$0' AND [ColumnName] = '$1' AND [TableNam...	DBTabelle	DBField	HydraAcronym
☐	ServiceParameter for ServiceParameterGui	ServiceParametersGui	ServiceParameters	[Domain] = '$0' AND [Service] = '$1' AND [Acronym] = '$2'	Domain	ServiceGui	Acronym
☑	ServiceParameters for Service	Services	ServiceParameters	[Parent] = '$id'			
☑	ServiceParametersGui for Service	Services	ServiceParametersGui	[DomainSet] = '$0' and [Domain] = '$1' AND [ServiceGui] = '$2'	DomainSet	Domain	Name
☐	ServiceParametersGui for ServiceGui	ServicesGui	ServiceParametersGui	[Parent] = '$id'			
☑	Services for Domain	Domains	Services	[Parent] = '$id->Services'			
☐	Service forServiceGui	ServicesGui	Services	[Name] = '$0'	Name		
☑	ServicesGui for Domain	Domains	ServicesGui	[Parent] = '$id->ServicesGui'			

The following columns are displayed:

Active

This checkbox specifies if the relation is used.

Name

The name of the relation – free choice.

Source

The table and its selection that are used to set the filters. If you edit an entry in this column, the currently possible assignments, i.e. all currently existing views are presented in a selection box.

Target

The table where the filter is applied. If you edit an entry in this column, the currently possible assignments, i.e. all currently existing views are presented in a selection box.

Filter

You can store the filter expression here that you want to apply to the target table. Variables ranging between $0 and $9 are supported. These variables are dynamically filled with the values of the columns Var[0-9].

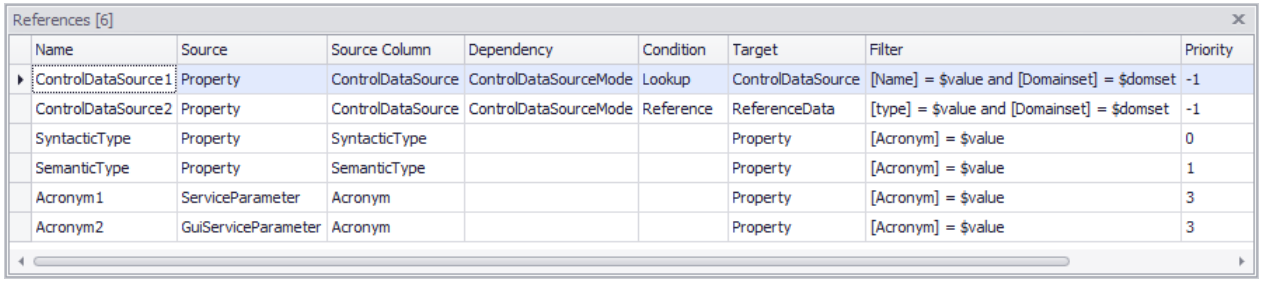
Var[0-9]

In these columns, you can specify the columns of the source table that are used to adapt the filters dynamically.

As soon as a correct (and activated) relation is entered in this table, it is applied. If you close one of the referenced views of a relation, the relevant view is removed from the relation. If this results in a double entry in the *Relations* table, this entry is removed. You can therefore use this view to administer the relations between concrete table instances and to administer unbound relations that can serve as template for relations.

1.8 References

References show the inherent connections between data records of the repository. They are defined by the repository structure and cannot be edited in the Repository Client. For example: A value in the column "Syntactic Type" of the *Property* table references another data record in the *Property* table. The *References* table lists the defined references and may be activated via the application menu (*Repository* → *References*).


References [6]

Name	Source	Source Column	Dependency	Condition	Target	Filter	Priority
ControlDataSource1	Property	ControlDataSource	ControlDataSourceMode	Lookup	ControlDataSource	[Name] = $value and [Domainset] = $domset	-1
ControlDataSource2	Property	ControlDataSource	ControlDataSourceMode	Reference	ReferenceData	[type] = $value and [Domainset] = $domset	-1
SyntacticType	Property	SyntacticType			Property	[Acronym] = $value	0
SemanticType	Property	SemanticType			Property	[Acronym] = $value	1
Acronym1	ServiceParameter	Acronym			Property	[Acronym] = $value	3
Acronym2	GuiServiceParameter	Acronym			Property	[Acronym] = $value	3

The following columns are displayed:

- **Name:** Name of the reference
- **Source**: The repository object type that can include this reference.
- **SourceColumn**: The source type property that can include the reference.
- **Dependency**: Source type property specifying the reference target.
- **Condition:** Value that the property specified under *Dependency* must have. Only then, the reference is pursued. For example: The value of ControlDataSourceMode (lookup, reference) specifies the target of the reference (ControlDataSource or ReferenceData) which is specified in the ControlDataSource property.
- **Target**: The repository object type that is referenced.

- **Filter:** Filter that selects the referenced data from the overall quantity of this type of data. Such a filter can include the variable $value (value of column *Value* in the current row) and $parent (value of column *Parent* in the current row).
- **Priority**: Specifies the priority of the reference. You can find further details in section "Get references".

References provide two general functions:

Show reference: You can use this function to display the referenced data in a new table. You can call this function via the context menu (*Context menu* → *Show reference*).

Note: This function is only available in the context menu of cells which can include references.

Get references: Use this function to complete missing values of a data record with values of referenced data records. For example: You can use this function to show the inherited values of a property of the SemanticType or the SyntacticType.

You can call this function via the context menu (*Context menu* → *Get references*) of the table views/grids. A new data record is generated and shown in a new panel. The generated data record is a copy of the currently selected data record. The values that are not filled are filled by those in the referenced data records. The reference priority specifies the filling sequence.

1.9 Service documentation

The Repository Client contains an extended documentation for selected standard services. The documentation mainly includes services released in the system and to be used with the Service Interface (SCS-SIF).

The documentation is available as of MRC version 1.8.STD.65500 (beginning of 2019).

There are two options to access the service documentation:

- In the toolbar "Repository", you can use the button *Service Documentation* to open the table of contents of the services included. Via hyperlinks, you can navigate to the different services.
- In the table views "Services" and "ServicesGui", you can use the context menu to open the documentation of the selected service if it is available.

Printing a service documentation:

You can print the service documentation using the shortcut Ctrl-P.